

Demonstrating SAGE:

A Dataflow-Native Framework for Modular, Controllable, and Transparent LLM-Augmented Reasoning

Jun Liu · Shuhao Zhang

Huazhong University of Science and Technology

01 Why pipeline-native execution?

LLM applications mix retrieval, memory, tools and generation with different runtime needs. Service-local controls obscure cross-stage queues, placement and performance.

Challenge

Make the whole application pipeline a manageable runtime object.

02 SAGE's contribution

Modular

Compose operators into explicit dataflows.

Controllable

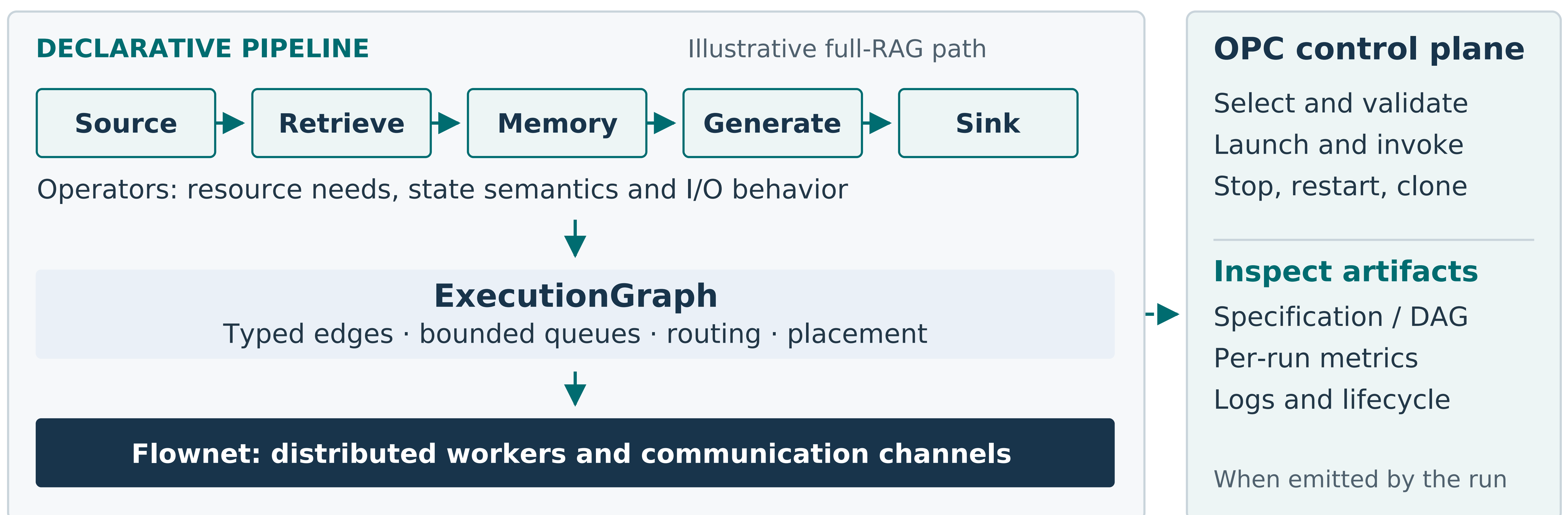
Compile pipelines into managed execution.

Transparent

Connect executions to inspectable artifacts.

03 From application logic to distributed execution

The pipeline remains explicit through specification, lowering and execution.



Runtime structure and coarse run evidence; no claim of complete per-record tracing.

04 What the audience explores

An OPC application is the entry point.

The SAGE pipeline is the object of inspection.

1 Select & validate

Resolve an entry's launch contract.

2 Launch & invoke

Exercise a representative request.

3 Inspect & control

Relate the graph, result and run artifacts.

Request classes in the paper

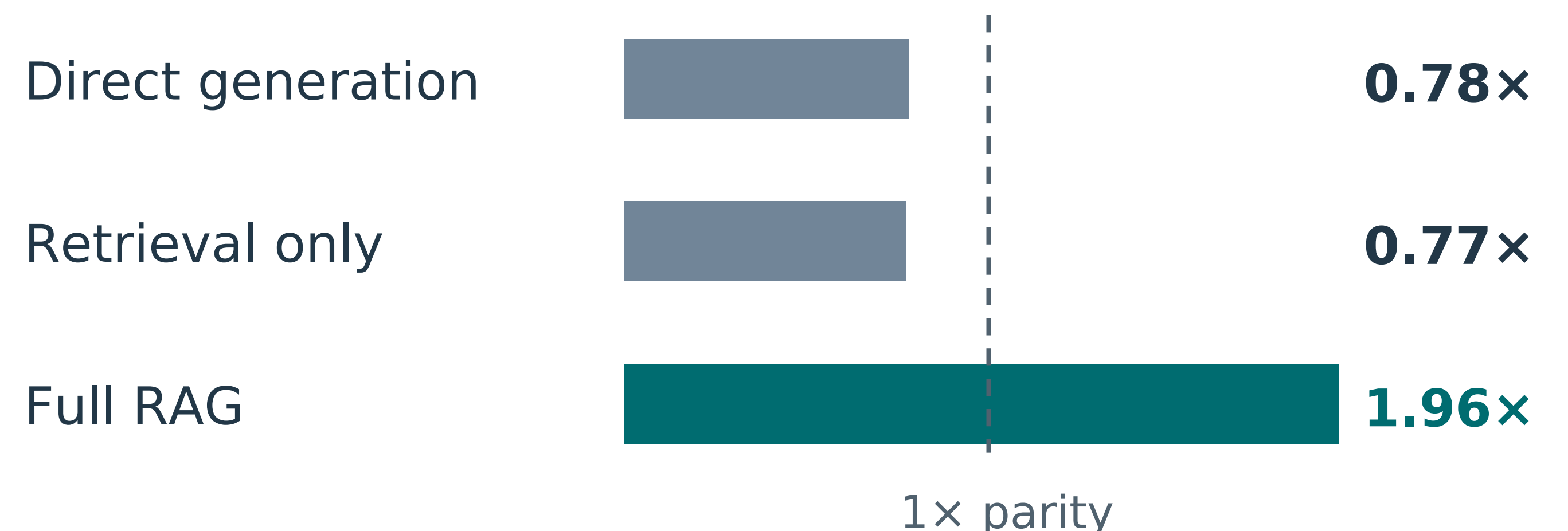
Retrieval-only · Tool-backed · Follow-up

Shared lifecycle, application-specific contracts.

05 Evidence from the paper

Throughput ratio: SAGE / LangChain RPC

8 nodes · 15 workloads · matched parallelism 64



Full RAG: 2.161 vs 1.101 queries/s.

Trade-off: mean latency 27.0 s vs 17.7 s.

Lighter paths remain below throughput parity.

32 requests/s; query-weighted micro aggregate.

Matched models and inputs; index build excluded.

TAKEAWAY Make LLM application pipelines explicit, manageable and inspectable.

The measured advantage is full-pipeline throughput, with workload-dependent costs.

